

Hard link vs. soft link

Part A - Create the links

Create a directory named link_lab and navigate to it.

```
(praneetha@Praneetha)-[~/Desktop]
$ mkdir link_lab

(praneetha@Praneetha)-[~/Desktop]
$ cd link_lab

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ pwd
/home/praneetha/Desktop/link_lab
```

Create a file named security.txt containing the following text:

Linux File System Security Lab

Confidential Security Report

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat > security.txt
Linux File System Security Lab
Confidential Security Report

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
```

Create:

a hard link named security_hard.txt

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ln security.txt security_hard.txt
```

a symbolic link named security_soft.txt

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ln -s security.txt security_soft.txt
```

List all three files using a command that displays their inode numbers.

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -li
total 8
1710576 -rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 13:46 security_soft.txt -> security.txt
1710576 -rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security.txt
```

Record the inode numbers of:

security.txt: 1710576

security_hard.txt: 1710576

security_soft.txt: 1710577

What difference do you observe?

security.txt and security_hard.txt have the same inode number, while security_soft.txt has a different inode number. This is because a hard link refers to the same inode as the original file, whereas a symbolic link is a separate file containing the target pathname.

Part B - Examine File Metadata

Use `ls -l` to examine the three files. Identify the file type of each file from the first character of the permission field.

```
(praneetha@Praneetha) - [~/Desktop/link_lab]
$ ls -l
total 8
-rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security_hard.txt
lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 13:46 security_soft.txt -> security.txt
-rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security.txt
```

- security.txt: Regular file
- security_hard.txt: Regular file
- security_soft.txt: Symbolic link

Compare the link count of the original file, hard link, and symbolic link. Explain the observed values.

```
(praneetha@Praneetha) - [~/Desktop/link_lab]
$ ls -li
total 8
1710576 -rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 13:46 security_soft.txt -> security.txt
1710576 -rw-rw-r-- 2 praneetha praneetha 60 Sep  6 13:46 security.txt
```

- security.txt: link count 2
 - security_hard.txt: link count 2
 - security_soft.txt: link count 1
- security.txt and security_hard.txt are two directory entries referring to the same inode, so the inode's link count is 2. The symbolic link is a separate inode, so its own link count is 1.

Use the `stat` command on all three files and record the information:

```
(praneetha@Praneetha) - [~/Desktop/link_lab]
$ stat security.txt
File: security.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1710576   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/praneetha)   Gid: ( 1000/praneetha)
Access: 2026-09-06 13:46:31.683136415 +0530
Modify: 2026-09-06 13:46:18.999097473 +0530
Change: 2026-09-06 13:46:38.959158329 +0530
Birth: 2026-09-06 13:46:05.827056009 +0530

(praneetha@Praneetha) - [~/Desktop/link_lab]
$ stat security_hard.txt
File: security_hard.txt
Size: 60          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1710576   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/praneetha)   Gid: ( 1000/praneetha)
Access: 2026-09-06 13:46:31.683136415 +0530
Modify: 2026-09-06 13:46:18.999097473 +0530
Change: 2026-09-06 13:46:38.959158329 +0530
Birth: 2026-09-06 13:46:05.827056009 +0530

(praneetha@Praneetha) - [~/Desktop/link_lab]
$ stat security_soft.txt
File: security_soft.txt -> security.txt
Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,1      Inode: 1710577   Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/praneetha)   Gid: ( 1000/praneetha)
Access: 2026-09-06 13:46:57.411212537 +0530
Modify: 2026-09-06 13:46:47.619184008 +0530
Change: 2026-09-06 13:46:47.619184008 +0530
Birth: 2026-09-06 13:46:47.619184008 +0530
```

Parameter	Original	Hard Link	Soft Link
Inode number	1710576	1710576	1710577
File type	Regular file	Regular file	Symbolic link
Size	60 bytes	60 bytes	12 bytes
Link count	2	2	1
Permissions	-rw-rw-r--	-rw-rw-r--	lrwxrwxrwx

Compare the file sizes of the original file, hard link, and symbolic link. Why might the size of the symbolic link differ from the original file?

The original file and hard link have the same file size because they refer to the same underlying inode and data. The symbolic link has a different, smaller size because it is a separate file that stores the pathname of the target, such as security.txt, rather than storing the actual file contents.

Part C - Investigate readlink

Execute readlink on security_soft.txt. What information is displayed?

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ readlink security_soft.txt
security.txt
```

readlink displays the pathname stored inside the symbolic link. In this case, the symbolic link stores security.txt as its target.

Execute readlink on security_hard.txt. What do you observe?

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ readlink security_hard.txt
```

No output is displayed because security_hard.txt is a hard link, not a symbolic link. A hard link does not store a target pathname, it directly refers to the same inode as the original file.

Based on your observations, explain why readlink works differently for hard links and symbolic links.

A symbolic link is a separate file that stores a pathname pointing to another file, so readlink can display that pathname. A hard link does not store a pathname to another file, it is another directory entry for the same inode. Therefore, readlink produces no output for a hard link.

Use:

readlink -f security_soft.txt

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ readlink -f security_soft.txt
/home/praneetha/Desktop/link_lab/security.txt
```

What does the -f option reveal?

The -f option resolves the symbolic link and displays the absolute path of the final target file.

Part D - Content Modification

Append the following line to security.txt:

Unauthorized login detected

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ echo "Unauthorized login detected" >> security.txt
```

Display the contents using all three filenames.

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
```

Is the newly added content visible through both links? Explain why.

Yes, the newly added content is visible through both the hard link and the symbolic link. The hard link and security.txt refer to the same inode, while the symbolic link points to security.txt. Therefore, both links access the updated file contents.

Modify the file using security_hard.txt. Check the contents through security.txt and security_soft.txt. What do you observe?

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ echo "Security investigation started" >> security_hard.txt

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security_soft.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

The modification is visible through both security.txt and security_soft.txt. This is because security_hard.txt and security.txt refer to the same inode, and security_soft.txt points to security.txt.

Part E - Rename Experiment

Rename security.txt to incident.txt

```
(praneetha@Praneetha)~[/Desktop/link_lab]
$ mv security.txt incident.txt

(praneetha@Praneetha)~[/Desktop/link_lab]
$ ls -li
total 8
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 incident.txt
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha  12 Sep  6 13:46 security_soft.txt → security.txt
```

Try accessing the content through security_hard.txt. Does it still work?

```
(praneetha@Praneetha)~[/Desktop/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

Yes. The hard link still works after the original file is renamed because both incident.txt and security_hard.txt refer to the same inode.

Try accessing the content through security_soft.txt. Does it still work?

```
(praneetha@Praneetha)~[/Desktop/link_lab]
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

No. The symbolic link does not work because it still points to security.txt, which no longer exists after the rename.

Use readlink security_soft.txt after renaming the original file. What path is still stored in the symbolic link?

```
(praneetha@Praneetha)~[/Desktop/link_lab]
$ readlink security_soft.txt
security.txt
```

The symbolic link still stores the pathname security.txt, even though the actual file has been renamed to incident.txt.

Explain why renaming the original file affects the symbolic link differently from the hard link.

Renaming the original file changes its directory name but does not change its inode. Therefore, the hard link continues to access the same inode and data. A symbolic link stores the target pathname rather than the inode. Since it still contains security.txt after the file was renamed to incident.txt, the symbolic link becomes broken.

Part F - Deletion Experiment

Recreate the links if necessary and then perform the following.

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ rm security_soft.txt

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ln -s incident.txt security_soft.txt

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -li
total 8
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 incident.txt
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 14:39 security_soft.txt -> incident.txt
```

Delete the original file.

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ rm incident.txt

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -li
total 4
1710576 -rw-rw-r-- 1 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 14:39 security_soft.txt -> incident.txt
```

Check whether security_hard.txt can still be accessed.

Yes, it can be accessed

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security_hard.txt
Linux File System Security Lab
Confidential Security Report
Unauthorized login detected
Security investigation started
```

Check whether security_soft.txt can still be accessed.

No, it cannot be accessed

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ cat security_soft.txt
cat: security_soft.txt: No such file or directory
```

Use ls -l and readlink to investigate the symbolic link after deleting its target.

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -l
total 4
-rw-rw-r-- 1 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 14:39 security_soft.txt -> incident.txt

(praneetha@Praneetha)-[~/Desktop/link_lab]
$ readlink security_soft.txt
incident.txt
```

What is a dangling/broken symbolic link?

A dangling or broken symbolic link is a symbolic link whose target file or directory no longer exists. The link remains, but it cannot access the target.

Does deleting the original filename immediately delete the underlying data when a hard link exists? Justify your answer using the link count and inode.

Before removing incident.txt

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -li
total 8
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 incident.txt
1710576 -rw-rw-r-- 2 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 14:39 security_soft.txt -> incident.txt
```

After removing incident.txt

```
(praneetha@Praneetha)-[~/Desktop/link_lab]
$ ls -li
total 4
1710576 -rw-rw-r-- 1 praneetha praneetha 119 Sep  6 14:29 security_hard.txt
1710577 lrwxrwxrwx 1 praneetha praneetha 12 Sep  6 14:39 security_soft.txt -> incident.txt
```

No. In our experiment, incident.txt and security_hard.txt initially had the same inode **1710576** and a link count of **2**. After deleting incident.txt, security_hard.txt still had inode **1710576**, but the link count decreased to **1**. Therefore, the underlying data was not deleted because the hard link still referred to the inode.

Part G - Directory and Filesystem Behaviour

Create a directory named evidence. Attempt to create a hard link to this directory. Record the result.

```
(praneetha@Praneetha)-[~/Desktop]
$ mkdir evidence

(praneetha@Praneetha)-[~/Desktop]
$ ln evidence evidence_hard
ln: evidence: hard link not allowed for directory
```

The operation failed because Linux generally does not allow users to create hard links to directories.

Create a symbolic link named evidence_link pointing to the evidence directory. Was the operation successful?

```
(praneetha@Praneetha)-[~/Desktop]
$ ln -s evidence evidence_link

(praneetha@Praneetha)-[~/Desktop]
$ ls -l
total 56
drwxrwxr-x 3 praneetha praneetha 4096 Aug 25 18:30 3_lab_support_files
drwxrwxr-x 2 praneetha praneetha 4096 Aug 13 18:37 assignment_1
drwxrwxr-x 2 praneetha praneetha 4096 Sep  6 14:58 evidence
lrwxrwxrwx 1 praneetha praneetha    8 Sep  6 15:00 evidence_link -> evidence
-rw-rw-r-- 1 praneetha praneetha    0 Aug 18 23:13 file1.txt
-rw-rw-r-- 1 praneetha praneetha    0 Aug 18 23:05 file.txt
drwxrwxr-x 2 praneetha praneetha 4096 Sep  6 14:42 link_lab
drwxrwxr-x 2 praneetha praneetha 4096 Aug 13 21:25 Linux
drwxrwxr-x 2 praneetha praneetha 4096 Aug 20 19:14 Logs
drwxrwxr-x 3 praneetha praneetha 4096 Aug 11 16:08 MSIS
drwxrwxr-x 2 praneetha praneetha 4096 Sep  1 20:11 ownership_lab
drwxrwxr-x 2 praneetha praneetha 4096 Aug 19 22:23 practice
drwxrwxr-x 2 praneetha praneetha 4096 Aug 19 22:25 practice_2
drwxrwxr-x 2 praneetha praneetha 4096 Aug 19 22:32 practice_3
drwxrwxr-x 2 praneetha praneetha 4096 Aug 19 22:36 practice_4
drwx----- 2 praneetha praneetha 4096 Aug 22 23:00 sample_dir
-rwx----- 1 praneetha praneetha 11 Aug 22 22:57 sample.txt
```

Yes, the operation was successful. A symbolic link can point to a directory.

Based on your experiment, compare hard and symbolic links with respect to directory linking.

Hard links to directories are generally not allowed because they could create directory cycles and complicate filesystem management. Symbolic links can point to directories because they simply store the target pathname.

If multiple filesystems/mount points are available in the laboratory environment, attempt to create:

```
(praneetha@Praneetha)-[~/Desktop]
$ df -T
Filesystem      Type      1K-blocks    Used Available Use% Mounted on
udev            devtmpfs   1969876        0   1969876   0% /dev
tmpfs           tmpfs      401136       1004    400132   1% /run
/dev/sda1       ext4      38818336 18312668 18501600 50% /
tmpfs           tmpfs      2005680         4    2005676   1% /dev/shm
tmpfs           tmpfs      2005680       248    2005432   1% /tmp
Kali            vboxsf    211430396 200754492 10675904 95% /media/sf_Kali
none            none        1024          0       1024    0% /run/credentials/getty@tty1.service
tmpfs           tmpfs      401136       108    401028   1% /run/user/1000
none            tmpfs       1024          0       1024    0% /run/credentials/systemd-journald.service
```

a hard link across filesystems

```
(praneetha@Praneetha)-[~/Desktop]
$ echo "Cross filesystem test" > cross_test.txt

(praneetha@Praneetha)-[~/Desktop]
$ ln cross_test.txt /media/sf_Kali/cross_hard.txt
ln: failed to create hard link '/media/sf_Kali/cross_hard.txt' => 'cross_test.txt': Invalid cross-device link
```

a symbolic link across filesystems

```
(praneetha@Praneetha)-[~/Desktop]
$ ln -s /home/praneetha/Desktop/cross_test.txt /media/sf_Kali/cross_soft.txt
ln: failed to create symbolic link '/media/sf_Kali/cross_soft.txt': Operation not permitted
```

A hard link attempt across the ext4 and vboxsf filesystems failed with “Invalid cross-device link.” A symbolic link attempt also failed with “Operation not permitted” because the VirtualBox shared-folder filesystem (vboxsf) in this environment does not permit the operation.

Final Comparison

Based only on your experimental observations, complete the following table:

Parameter	Hard Link	Symbolic Link
Inode number	Same as original file	Different from original file
File type	Regular file (-)	Symbolic link (l)
Link count	Increases link count of the same inode	Usually 1 for the symbolic-link inode
File size	Same as original file	Size of the stored target pathname
Stores target pathname?	No	Yes
readlink output	No output	Displays target pathname
Effect of modifying target	Changes are visible	Changes are visible
Effect of renaming target	Still works	Becomes broken if pathname changes
Effect of deleting target	Still works if another hard link exists	Becomes broken/dangling
Can become a dangling or broken link?	No, not in the normal symbolic-link sense	Yes
Directory linking	Not allowed in my experiment	Allowed
Cross-filesystem linking	Failed: Invalid cross-device link	Failed in my vboxsf environment with ‘Operation not permitted’

sort

Suppose alerts.txt contains:

192.168.1.10 admin 5

10.0.0.8 root 12

172.16.0.5 guest 2

10.0.0.20 admin 8

Format:

IP-address username failed attempts

```
(praneetha@Praneetha)-[~/Desktop]
$ cat > alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8

(praneetha@Praneetha)-[~/Desktop]
$ cat alerts.txt
192.168.1.10 admin 5
10.0.0.8 root 12
172.16.0.5 guest 2
10.0.0.20 admin 8
```

Sort according to failed attempts

```
(praneetha@Praneetha)-[~/Desktop]
$ sort -k3,3n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Sort a list of suspicious IP addresses alphabetically.

```
(praneetha@Praneetha)-[~/Desktop]
$ cut -d' ' -f1 alerts.txt | sort
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Display only unique suspicious IP addresses using sort.

```
(praneetha@Praneetha)-[~/Desktop]
$ cut -d' ' -f1 alerts.txt | sort -u
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Sort a list of detected port numbers numerically.

```
(praneetha@Praneetha)-[~/Desktop]
$ printf "443\n22\n8080\n80\n21\n" | sort -n
21
22
80
443
8080
```

Display port numbers in descending numerical order.

```
(praneetha@Praneetha)-[~/Desktop]
$ printf "443\n22\n8080\n80\n21\n" | sort -nr
8080
443
80
22
21
```

Sort usernames without considering uppercase/lowercase differences.

```
(praneetha@Praneetha)-[~/Desktop]
$ cut -d' ' -f2 alerts.txt | sort -f
admin
admin
guest
root
```

From a failed-login log, count how many times each IP address occurs and display the most frequent IP first.

```
(praneetha@Praneetha)-[~/Desktop]
$ cut -d' ' -f1 alerts.txt | sort | uniq -c | sort -nr
1 192.168.1.10
1 172.16.0.5
1 10.0.0.8
1 10.0.0.20
```

Given IP,username,failed_attempts, sort records based on the number of failed attempts.

```
(praneetha@Praneetha)-[~/Desktop]
$ sort -k3,3n alerts.txt
172.16.0.5 guest 2
192.168.1.10 admin 5
10.0.0.20 admin 8
10.0.0.8 root 12
```

Sort a CSV vulnerability report based on severity score.

```
(praneetha@Praneetha)~[/Desktop]
$ printf "V001,SQL Injection,9\nV002,XSS,6\nV003,Buffer Overflow,10\nV004,Weak Password,3\n" > vulnerability.csv

(praneetha@Praneetha)~[/Desktop]
$ cat vulnerability.csv
V001,SQL Injection,9
V002,XSS,6
V003,Buffer Overflow,10
V004,Weak Password,3

(praneetha@Praneetha)~[/Desktop]
$ sort -t',' -k3,3n vulnerability.csv
V004,Weak Password,3
V002,XSS,6
V001,SQL Injection,9
V003,Buffer Overflow,10
```

Extract IP addresses using grep, then use sort and uniq to identify unique IP addresses.

```
(praneetha@Praneetha)~[/Desktop]
$ grep -oE '[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+' alerts.txt | sort | uniq
10.0.0.20
10.0.0.8
172.16.0.5
192.168.1.10
```

Create a pipeline that displays the top 5 IP addresses responsible for the most failed login attempts.

```
(praneetha@Praneetha)~[/Desktop]
$ sort -k3,3nr alerts.txt | head -5 | cut -d' ' -f1
10.0.0.8
10.0.0.20
192.168.1.10
172.16.0.5
```